

TNSVT

Mentoria de Elite - Eleva tu Trading

Plan de Implementacion

De proyecto local a producto escalable (50 personas)

Version	1.0
Fecha	Junio 2026
Estado	Aprobado para implementacion
Hosting elegido	Hostinger VPS KVM 2
Inversion inicial	~\$135 USD (ARS 162.000)
Proyeccion 12m	50 clientes / \$500+ USD/mes

Documento interno y confidencial

Tabla de Contenidos

- 1** Resumen ejecutivo
- 2** Producto y pricing
- 3** Infraestructura elegida
- 4** Comparativa de opciones
- 5** Plan de ejecución técnico
- 6** Setup del VPS
- 7** Deploy de la aplicación
- 8** Cloudflare Tunnel y HTTPS
- 9** Migración a MySQL
- 10** Integración MercadoPago
- 11** Sistema de trial 14 días
- 12** Backups automatizados
- 13** Monitoreo y alertas
- 14** Rebuild del APK
- 15** Cálculos económicos
- 16** Proyecciones a 50 personas
- 17** Plan de contingencia
- 18** Cronograma de 12 meses
- 19** Checklist de implementación
- 20** Glosario técnico

Plan de Implementación TNSVT

De proyecto local a producto escalable (50 personas)

Versión: 1.0

Fecha: Junio 2026

Autor: TNSVT

Estado: Aprobado para implementación

Tabla de contenidos

- [Resumen ejecutivo](#1-resumen-ejecutivo)
- [Producto y pricing](#2-producto-y-pricing)
- [Infraestructura elegida](#3-infraestructura-elejida)
- [Comparativa de opciones](#4-comparativa-de-opciones)
- [Plan de ejecución técnico](#5-plan-de-ejecución-técnico)
- [Setup del VPS](#6-setup-del-vps)
- [Deploy de la aplicación](#7-deploy-de-la-aplicación)
- [Cloudflare Tunnel y HTTPS](#8-cloudflare-tunnel-y-https)
- [Migración a MySQL](#9-migración-a-mysql)
- [Integración MercadoPago](#10-integración-mercadopago)
- [Sistema de trial 14 días](#11-sistema-de-trial-14-días)
- [Backups automatizados](#12-backups-automatizados)
- [Monitoreo y alertas](#13-monitoreo-y-alertas)
- [Rebuild del APK](#14-rebuild-del-apk)
- [Cálculos económicos](#15-cálculos-económicos)
- [Proyecciones a 50 personas](#16-proyecciones-a-50-personas)
- [Plan de contingencia](#17-plan-de-contingencia)
- [Cronograma de 12 meses](#18-cronograma-de-12-meses)
- [Checklist de implementación](#19-checklist-de-implementación)
- [Glosario técnico](#20-glosario-técnico)

1. Resumen ejecutivo

TNSVT es una aplicación de mentoría de trading con app móvil nativa Android (APK) y PWA. Actualmente corre en una PC local con Tailscale para acceso privado. El objetivo es transformarlo en un producto comercial escalable para 50 clientes pagos.

Decisiones clave

Decisión	Elección
Hosting	Hostinger VPS KVM 2
Dominio	.app (profesional, HTTPS forzado)
HTTPS	Cloudflare Tunnel (gratis)
Base de datos	MySQL/MariaDB (no SQLite)
Pasarela de pagos	MercadoPago (AR)
Pricing	\$9 USD/mes base + upsells escalonados
Trial	14 días gratis + recordatorios
Backups	Diarios a Cloudflare R2
Monitoreo	UptimeRobot + Sentry

Inversión inicial

Concepto	Costo
VPS Hostinger KVM 2 (12 meses)	~\$120 USD (ARS 144.000)
Dominio .app (1 año)	\$15 USD (ARS 18.000)
Total primer año	~\$135 USD (ARS 162.000)
Equivalente mensual	~\$11 USD/mes (ARS 13.500)

Proyección a 12 meses

Mes	Clientes	Ingreso/mes	Ganancia neta/mes
1-2	0-3	\$0-30	-\$11 a +\$19
3-4	5-10	\$50-100	+\$39 a +\$89
5-6	15-25	\$150-250	+\$139 a +\$239
7-9	30-40	\$300-400	+\$289 a +\$389
10-12	45-50	\$450-500	+\$439 a +\$489

2. Producto y pricing

Modelo de pricing escalonado

Concepto	Precio
Plan Base	\$9 USD/mes
- Acceso a academia	incluido
- Chat con mentores	incluido
- Journal de trading	incluido
- Notificaciones in-app	incluido
- Comunidad general	incluido
Upsell 1: Señales	+\$5 USD/mes
- Señales diarias de trading	
- Análisis de mercado en vivo	
- Alertas de oportunidades	
Upsell 2: Mentoría 1:1	+\$20 USD/mes
- Sesión semanal 1:1 con mentor	

Concepto	Precio
- Revisión de trades personalizada	
- Plan de trading a medida	
- WhatsApp directo con mentor	
Upsell 3: Cursos premium	\$50-200 USD (pago único)
- Curso avanzado de price action	
- Curso de psicología de trading	
- Masterclass de gestión de riesgo	

Ingreso promedio por cliente

Combinación	Ingreso/mes
Solo base	\$9
Base + Señales	\$14
Base + Mentoría	\$29
Base + Señales + Mentoría	\$34
Promedio realista	\$15-20/mes

Proyecciones conservadoras con upsells

Clientes	Mix estimado	Ingreso promedio	Ingreso/mes
10	5 base / 3 base+señales / 2 base+mentoría	\$15.50	\$155
25	12 / 8 / 5	\$15.40	\$385
50	20 / 18 / 12	\$17.50	\$875

3. Infraestructura elegida

Hostinger VPS KVM 2 - Especificaciones

Spec	Valor
CPU	2 vCPU (AMD EPYC)
RAM	8 GB DDR4
Almacenamiento	100 GB NVMe SSD
Ancho de banda	8 TB/mes
OS	Ubuntu 24.04 LTS
Datacenter	US East (Ashburn, VA)
IPv4	1 incluida
IPv6	/64 incluido
Acceso root	Sí (SSH)
Soporte	24/7 en español
Panel	hPanel con terminal web
Backups automatizados	+20% (opcional)
Costo	\$9.99/mes (12m) / \$14.99/mes renew

Capacidad estimada

Métrica	Capacidad
Usuarios registrados	500-1000
Concurrentes en hora pico	50-80
Requests por minuto (pico)	1000-2000
Almacenamiento para DB	100 GB (sobra)
Ancho de banda	8 TB/mes (sobra)

4. Comparativa de opciones

Web Hosting compartido vs VPS vs Dedicado

Aspecto	Web Hosting	VPS (elegido)	Dedicado
Costo mensual	\$2.99-15.99	\$9.99	\$80+
Control root	■	■	■
Accesible por SSH	Limitado	■ Completo	■ Completo
Instalar Composer	■■ Limitado	■	■
Instalar paquetes	■	■	■
Cloudflare Tunnel	■	■	■
Push FCM workers	■	■	■
Procesos background	■	■	■
SQLite	■■ Problemas	■	■
MySQL	■	■	■
Performance 50 users	■ Se cae	■ OK	■ Sobra
Escalabilidad	■	■■ Media	■ Alta

Proveedores VPS evaluados

Proveedor	Plan	Costo/mes	Soporte	Datacenter cerca AR	Veredicto
Hostinger KVM 2	2vCPU/8GB	\$9.99	24/7 ES	■ US East	■ Elegido
Hetzner CX32	4vCPU/8GB	€5.99 (~\$6.4)	■ Community	■ US East	Más barato, sin soporte

Proveedor	Plan	Costo/mes	Soporte	Datacenter cerca AR	Veredicto
DigitalOcean 8GB	2vCPU/8GB	\$24	24/7 EN	■ US East	Más caro, mejor UI
Vultr 8GB	4vCPU/8GB	\$24	24/7 EN	■ US East	Similar a DO
Linode 8GB	4vCPU/8GB	\$24	24/7 EN	■ US East	Similar

Veredicto: Hostinger por soporte 24/7 en español, pago en ARS, y hPanel para tareas comunes.

5. Plan de ejecución técnico

Resumen de fases

Fase	Duración	Tareas
0. Compra	30 min	Comprar VPS, dominio, configurar SSH
1. Setup server	45 min	Instalar PHP, Nginx, MySQL, Composer
2. Deploy código	30 min	Clonar repo, instalar deps, migrar DB
3. HTTPS	20 min	Configurar Cloudflare Tunnel
4. MercadoPago	60 min	Crear cuenta, integrar SDK, webhooks
5. Trial system	30 min	Implementar 14 días + recordatorios
6. Backups	20 min	Script + cron + R2
7. Monitoreo	15 min	UptimeRobot + Sentry
8. APK rebuild	30 min	Cambiar URLs, build, firma, instalar
9. Testing E2E	30 min	Probar todo el flujo de pago y trial

Fase	Duración	Tareas
Total	~5 horas	

6. Setup del VPS

6.1. Compra del VPS

- Ir a <https://www.hostinger.com/vps-hosting>
- Elegir **KVM 2** (2vCPU, 8GB RAM, 100GB NVMe)
- Plan: **12 meses** (más barato)
- OS: **Ubuntu 24.04 LTS**
- Datacenter: **US East (Ashburn, VA)** — más cerca de Argentina
- Hostname: **tnsvt-prod**
- Pagar con MercadoPago / tarjeta / Binance
- Recibir email con: IP pública, password root, panel hPanel

6.2. Primer acceso SSH

```
ssh root@TU_IP_VPS # Ingresar password recibido por email
```

6.3. Hardening básico

```
# Crear usuario deployer (no usar root) adduser deployer usermod -aG sudo deployer #
Configurar SSH key para deployer mkdir -p /home/deployer/.ssh cp ~/.ssh/authorized_keys
/home/deployer/.ssh/ chown -R deployer:deployer /home/deployer/.ssh chmod 700
/home/deployer/.ssh chmod 600 /home/deployer/.ssh/authorized_keys # Deshabilitar login
root sed -i 's/PermitRootLogin yes/PermitRootLogin no/' /etc/ssh/sshd_config systemctl
restart sshd
```

6.4. Instalar stack

```
# Actualizar sistema apt update && apt upgrade -y # Instalar PHP 8.4 y extensiones apt
install -y software-properties-common add-apt-repository ppa:ondrej/php apt update apt
install -y php8.4 php8.4-fpm php8.4-cli php8.4-mbstring \ php8.4-xml php8.4-curl
php8.4-mysql php8.4-sqlite3 \ php8.4-intl php8.4-zip php8.4-bcmath php8.4-openssl #
Instalar Nginx apt install -y nginx certbot python3-certbot-nginx # Instalar MariaDB apt
install -y mariadb-server mysql_secure_installation # Instalar Composer curl -sS
https://getcomposer.org/installer | php mv composer.phar /usr/local/bin/composer #
Instalar Node.js (para build de assets) curl -fsSL https://deb.nodesource.com/setup_20.x
| bash - apt install -y nodejs # Instalar git, unzip, sqlite3 apt install -y git unzip
sqlite3
```

7. Deploy de la aplicación

7.1. Clonar el repo

```
# Como deployer sudo mkdir -p /var/www sudo chown deployer:deployer /var/www cd /var/www
git clone https://github.com/federicocasal73-gif/tnsvt-symfony.git tnsvt cd tnsvt
```

7.2. Instalar dependencias

```
# PHP dependencies composer install --no-dev --optimize-autoloader # JS dependencies
(para build de assets) npm install npm run build # Permisos sudo chown -R
deployer:www-data . sudo chmod -R 775 var/
```

7.3. Configurar variables de entorno

```
# Editar .env.local (NO commitear) cat > .env.local << 'EOF' APP_ENV=prod
APP_SECRET=GENERAR_CON_PHP_BIN_CONSOLE_SECRET DATABASE_URL="mysql://tnsvt_user:PASSWORD@
127.0.0.1:3306/tnsvt_prod?serverVersion=8.0.32&charset=utf8mb4"
CORS_ALLOW_ORIGIN="^https?:/(tnsvt\.app)(:[0-9]+)?$ APP_VERSION=1.3.0 APP_VERSION_CODE=4
APP_DOWNLOAD_URL="https://tnsvt.app/downloads/app-release.apk APP_RELEASE_NOTES="..."
APP_UPDATE_REQUIRED=false # MercadoPago MP_ACCESS_TOKEN=APP_USR-XXXX
MP_PUBLIC_KEY=APP_USR-XXXX # Email (SendGrid)
MAILER_DSN=smtp://apikey:SG.XXXXXX@smtp.sendgrid.net:587 MAILER_FROM=noreply@tnsvt.app #
Firebase FIREBASE_CREDENTIALS_PATH=var/firebase/service-account.json
FIREBASE_WEB_API_KEY=XXXX FIREBASE_AUTH_DOMAIN=tnsvt-app.firebaseio.com
FIREBASE_PROJECT_ID=tnsvt-app FIREBASE_MESSAGING_SENDER_ID=XXXX FIREBASE_APP_ID=XXXX EOF
```

7.4. Crear base de datos

```
sudo mysql

CREATE DATABASE tnsvt_prod CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci; CREATE USER
'tnsvt_user'@'localhost' IDENTIFIED BY 'PASSWORD_SEGURO'; GRANT ALL PRIVILEGES ON
tnsvt_prod.* TO 'tnsvt_user'@'localhost'; FLUSH PRIVILEGES; EXIT;
```

7.5. Migrar schema

```
php bin/console doctrine:migrations:migrate --no-interaction
```

7.6. Compilar assets

```
php bin/console asset-map:compile
```

7.7. Cache warmup

```
php bin/console cache:clear --env=prod php bin/console cache:warmup --env=prod
```

8. Cloudflare Tunnel y HTTPS

8.1. Crear cuenta Cloudflare

- Ir a <https://dash.cloudflare.com/sign-up>
- Crear cuenta con email
- Agregar dominio **tnsvt.app** (comprado en Namecheap/Cloudflare Registrar)

8.2. Instalar cloudflared en el VPS

```
curl -L --output cloudflared.deb https://github.com/cloudflare/cloudflared/releases/latest/download/cloudflared-linux-amd64.deb sudo dpkg -i cloudflared.deb
```

8.3. Login y crear tunnel

```
cloudflared tunnel login cloudflared tunnel create tnsvt-prod cloudflared tunnel route dns tnsvt-prod tnsvt.app cloudflared tunnel route dns tnsvt-prod www.tnsvt.app
```

8.4. Configurar tunnel

```
sudo mkdir -p /etc/cloudflared sudo nano /etc/cloudflared/config.yml
```

```
tunnel: tnsvt-prod credentials-file: /etc/cloudflared/<TUNNEL_ID>.json ingress: -  
hostname: tnsvt.app service: http://localhost:80 - hostname: www.tnsvt.app service:  
http://localhost:80 - service: http_status:404
```

8.5. Instalar como servicio

```
sudo cloudflared service install sudo systemctl enable cloudflared sudo systemctl start  
cloudflared sudo systemctl status cloudflared
```

8.6. Configurar Nginx

```
sudo nano /etc/nginx/sites-available/tnsvt
```

```
server { listen 80; server_name tnsvt.app www.tnsvt.app; root /var/www/tnsvt/public;  
index index.php; # Security headers add_header X-Content-Type-Options nosniff;
```

```

add_header X-Frame-Options DENY; add_header Referrer-Policy
"strict-origin-when-cross-origin"; add_header Permissions-Policy "geolocation=(),
microphone=(), camera=()"; # CSP (Content Security Policy) add_header
Content-Security-Policy "default-src 'self'; script-src 'self' 'unsafe-inline'
https://www.mercadopago.com https://*.firebaseio.com; style-src 'self' 'unsafe-inline'
https://fonts.googleapis.com; font-src 'self' https://fonts.gstatic.com; img-src 'self'
data: https:; connect-src 'self' https://fcm.googleapis.com
https://fcmregistrations.googleapis.com https://*.firebaseio.com
https://*.googleapis.com https://api.mercadopago.com wss: ws: http: https: data: blob:;
frame-src 'self' https://www.mercadopago.com https://*.firebaseio.com; frame-ancestors
'self';"; # PHP-FPM location ~ ^/index\.php(/|$) { fastcgi_pass
unix:/run/php/php8.4-fpm.sock; fastcgi_split_path_info ^(.+\.php)(/.*)$; include
fastcgi_params; fastcgi_param SCRIPT_FILENAME $realpath_root$fastcgi_script_name;
fastcgi_param DOCUMENT_ROOT $realpath_root; internal; } location ~ \.php$ { return 404;
} # Symfony routing location / { try_files $uri /index.php$is_args$args; } # Cache de
assets location ~* \.(js|css|png|jpg|jpeg|gif|ico|svg|woff|woff2|ttf|eot)$ { expires 1y;
add_header Cache-Control "public, immutable"; } }

sudo ln -s /etc/nginx/sites-available/tnsvt /etc/nginx/sites-enabled/ sudo nginx -t sudo
systemctl restart nginx

```

8.7. Habilitar proxy en Cloudflare

En el dashboard de Cloudflare:

- DNS: el registro A de **tnsvt.app** debe tener el proxy activado (naranja)
- SSL/TLS: modo "Full"
- Always Use HTTPS: ON
- Min TLS Version: 1.2

9. Migración a MySQL

9.1. Por qué migrar

Aspecto	SQLite	MySQL
Concurrencia	■ Una escritura a la vez	■ Multi-usuario
Backups	Copiar archivo .db	mysqldump o R2
Performance con 50 users	■ Se traba	■ OK
Producción	No recomendado	Estándar

9.2. Schema migration

Doctrine detecta el cambio automáticamente al apuntar a MySQL. Solo correr:

```
php bin/console doctrine:migrations:migrate --no-interaction
```

9.3. Migrar datos existentes (si hay)

```
# Exportar SQLite a SQL sqlite3 var/data_dev.db .dump > /tmp/dump.sql # Importar en
MySQL (con ajustes manuales de sintaxis) mysql -u tnsvt_user -p tnsvt_prod <
/tmp/dump.sql
```

10. Integración MercadoPago

10.1. Crear cuenta MercadoPago

- Ir a <https://www.mercadopago.com.ar/developers/panel>
- Crear aplicación: "TNSVT"
- Obtener credenciales:
- **Access Token** (servidor, secret)
- **Public Key** (cliente, public)
- Configurar webhooks

10.2. Instalar SDK PHP

```
composer require mercadopago/dx-php
```

10.3. Configurar variables de entorno

```
# .env.local MP_ACCESS_TOKEN=APP_USR-1234567890-XXXXXX-XXXX
MP_PUBLIC_KEY=APP_USR-XXXX-XXXX-XXXX
MP_NOTIFICATION_URL=https://tnsvt.app/api/mercadopago/webhook
```

10.4. Implementar checkout

Crear `src/Service/MercadoPagoService.php`:

```
namespace App\Service; use MercadoPago\SDK; use MercadoPago\Preference; use
MercadoPago\Item; class MercadoPagoService { public function
createSubscriptionPreference(string $plan, string $userId): array {
    SDK::setAccessToken($_ENV['MP_ACCESS_TOKEN']); $preference = new Preference(); $item =
    new Item(); $item->title = "TNSVT - Plan $plan"; $item->quantity = 1; $item->unit_price
    = $this->getPlanPrice($plan); $item->currency_id = 'ARS'; $preference->items = [$item];
```

```
$preference->external_reference = $userId; $preference->back_urls = [ 'success' =>
'https://tnsvt.app/mercado_pago/success', 'failure' =>
'https://tnsvt.app/mercado_pago/failure', 'pending' =>
'https://tnsvt.app/mercado_pago/pending', ]; $preference->auto_return = 'approved';
$preference->notification_url = $_ENV['MP_NOTIFICATION_URL']; $preference->save();
return [ 'id' => $preference->id, 'init_point' => $preference->init_point, ]; } public
function getPlanPrice(string $plan): float { return match($plan) { 'base' => 9 * 1100,
// $9 USD a ARS (cotización del día) 'senales' => 14 * 1100, 'mentoria' => 29 * 1100,
default => 0, }; } }
```

10.5. Webhook controller

Crear `src/Controller/Webhook/MercadoPagoWebhookController.php`:

```
namespace App\Controller\Webhook; use App\Service\MercadoPagoService; use
Symfony\Bundle\FrameworkBundle\Controller\AbstractController; use
Symfony\Component\HttpFoundation\Request; use Symfony\Component\HttpFoundation\Response;
use Symfony\Component\Routing\Attribute\Route; class MercadoPagoWebhookController
extends AbstractController { #[Route('/api/mercado_pago/webhook', name:
'mercado_pago_webhook', methods: ['POST'])] public function webhook(Request $request,
MercadoPagoService $mp): Response { $data = json_decode($request->getContent(), true);
// MercadoPago envía notificaciones de pagos if (isset($data['type']) && $data['type']
=== 'payment') { $paymentId = $data['data']['id']; $payment =
$mp->getPayment($paymentId); if ($payment && $payment->status === 'approved') { //
Activar suscripción del usuario $userId = $payment->external_reference;
$this->activateUserSubscription($userId, $payment); } } return new Response('OK', 200);
} private function activateUserSubscription(string $userId, $payment): void { // Lógica:
actualizar DB, enviar email, etc } }
```

10.6. Fees de MercadoPago

Tipo	Fee
Suscripción mensual	4.99% del monto
Transferencia a cuenta bancaria	Gratis (5-10 días)
Mantenimiento de cuenta	\$0
Comisión por venta con QR	0% (no aplica)

10.7. Facturación

- MercadoPago emite facturas automáticas (C o B según CUIT)
- Se pueden descargar desde el panel de MercadoPago
- Compatible con AFIP

11. Sistema de trial 14 días

11.1. Schema de DB

Agregar campos a tabla `user`:

- `trial_started_at` (datetime, nullable)
- `trial_ends_at` (datetime, nullable)
- `subscription_status` (enum: 'trial', 'active', 'expired', 'cancelled')
- `subscription_plan` (string, nullable)
- `subscription_started_at` (datetime, nullable)
- `subscription_ends_at` (datetime, nullable)
- `mp_subscription_id` (string, nullable)

11.2. Crear migración

```
php bin/console make:migration php bin/console doctrine:migrations:migrate --no-interaction
```

11.3. Lógica de trial

Crear `src/Service/TrialService.php`:

```
namespace App\Service; use App\Entity\User; use Doctrine\ORM\EntityManagerInterface; class TrialService { public function __construct(private EntityManagerInterface $em) {} public function startTrial(User $user): void { $now = new \DateTime(); $user->setTrialStartedAt($now); $user->setTrialEndsAt((clone $now)->modify('+14 days')); $user->setSubscriptionStatus('trial'); $this->em->flush(); } public function isTrialActive(User $user): bool { if ($user->getSubscriptionStatus() !== 'trial') { return false; } $now = new \DateTime(); return $user->getTrialEndsAt() > $now; } public function daysLeft(User $user): int { if (!$this->isTrialActive($user)) { return 0; } $now = new \DateTime(); $diff = $user->getTrialEndsAt()->diff($now); return $diff->days; } public function sendReminders(): void { // Cron que corre diario $users = $this->em->getRepository(User::class) ->findBy(['subscriptionStatus' => 'trial']); foreach ($users as $user) { $daysLeft = $this->daysLeft($user); if (in_array($daysLeft, [7, 3, 1])) { $this->sendReminderEmail($user, $daysLeft); } if ($daysLeft === 0) { $user->setSubscriptionStatus('expired'); $this->em->flush(); } } }
```

11.4. Email templates

- **Día 7:** "Te quedan 7 días de trial. ¿Querés suscribirte?"
- **Día 3:** "Te quedan 3 días. Suscribite por solo \$9/mes"
- **Día 1:** "Mañana vence tu trial. No te quedes afuera"
- **Día 0:** "Tu trial venció. Activá tu cuenta por \$9/mes"

12. Backups automatizados

12.1. Script de backup

```
sudo nano /usr/local/bin/backup-tnsvt.sh
```

```
#!/bin/bash FECHA=$(date +%Y%m%d-%H%M) BACKUP_DIR=/var/backups/tnsvt DB_NAME=tnsvt_prod  
DB_USER=tnsvt_user DB_PASS=PASSWORD mkdir -p $BACKUP_DIR # Backup DB mysqldump -u  
$DB_USER -p$DB_PASS $DB_NAME | gzip > $BACKUP_DIR/db-$FECHA.sql.gz # Backup archivos  
(uploads, var/) tar -czf $BACKUP_DIR/files-$FECHA.tar.gz -C /var/www/tnsvt var/ # Subir  
a Cloudflare R2 (requiere rclone configurado) rclone copy $BACKUP_DIR/db-$FECHA.sql.gz  
r2:tnsvt-backups/db/ rclone copy $BACKUP_DIR/files-$FECHA.tar.gz r2:tnsvt-backups/files/  
# Limpiar backups locales (mantener últimos 3) ls -t $BACKUP_DIR/db-*.sql.gz | tail -n  
+4 | xargs -r rm ls -t $BACKUP_DIR/files-*.tar.gz | tail -n +4 | xargs -r rm echo "Backup  
completado: $FECHA"
```

```
sudo chmod +x /usr/local/bin/backup-tnsvt.sh
```

12.2. Cron diario

```
sudo crontab -e # Agregar: 0 3 * * * /usr/local/bin/backup-tnsvt.sh >>  
/var/log/tnsvt-backup.log 2>&1
```

12.3. Cloudflare R2 setup

- Crear cuenta Cloudflare (ya tenés)
- R2 → Create bucket → **tnsvt-backups**
- Generar API token con permisos R2
- Configurar rclone:

```
sudo apt install rclone rclone config # Seguir wizard para R2
```

12.4. Costo R2

- Storage: \$0.015/GB/mes
- 100 MB/mes de backups = \$0.0015/mes ≈ \$0
- Prácticamente gratis

13. Monitoreo y alertas

13.1. UptimeRobot (gratis)

- Ir a <https://uptimerobot.com>
- Crear cuenta
- Add New Monitor:
- Type: HTTPS
- URL: <https://tnsvt.app>
- Interval: 5 minutes
- Configurar alertas:
- Email: tu email
- Telegram: opcional (más rápido)

13.2. Sentry (gratis hasta 5K eventos/mes)

- Ir a <https://sentry.io>
- Crear proyecto PHP
- Instalar SDK:

```
composer require sentry/sentry-symfony
```

- Configurar DSN en `.env.local`:

```
SENTRY_DSN=https://XXXX@sentry.io/XXXX
```

13.3. Logs centralizados

```
# Tail en vivo sudo tail -f /var/log/nginx/tnsvt.error.log sudo tail -f /var/www/tnsvt/var/log/prod.log
```

13.4. Health check endpoint

Crear `src/Controller/Api/HealthController.php`:

```
#[Route('/api/health', methods: ['GET'])] public function health(): JsonResponse {  
    return new JsonResponse([ 'status' => 'ok', 'time' => (new \DateTime())->format('c'),  
        'version' => $_ENV['APP_VERSION'] ?? 'unknown', ]); }  
}
```

14. Rebuild del APK

14.1. Cambiar URLs en código

`capacitor.config.json`:

```
{ "server": { "url": "https://tnsvt.app" } }
```

android/app/src/main/res/xml/network_security_config.xml:

```
<domain includeSubdomains="true">tnsvt.app</domain> <domain  
includeSubdomains="true">www.tnsvt.app</domain>
```

assets/api.js:

```
return 'https://tnsvt.app';
```

14.2. Compilar y firmar

```
.\build_apk.bat release
```

14.3. Subir APK a GitHub Releases

```
git tag v1.4.0 git push origin v1.4.0
```

En GitHub → Releases → Draft new release:

- Tag: v1.4.0
- Title: "TNSVT v1.4.0 - Producción"
- Subir `app-release.apk`
- Publish

14.4. Distribuir a clientes

Links de descarga:

- GitHub Releases (gratis, confiable)
- Google Drive (gratis, fácil de compartir)
- OneDrive / Dropbox
- Hostinger file manager

15. Cálculos económicos

15.1. Inversión inicial (primer año)

Concepto	USD	ARS (~\$1.200/USD)
VPS KVM 2 (12m)	\$120	ARS 144.000

Concepto	USD	ARS (~\$1.200/USD)
Dominio .app (1 año)	\$15	ARS 18.000
Total	\$135	ARS 162.000
Equivalente mensual	\$11.25	ARS 13.500

15.2. Costos mensuales recurrentes

Concepto	USD/mes	ARS/mes
VPS KVM 2 (renew)	\$14.99	ARS 17.988
Dominio (prorrateado)	\$1.25	ARS 1.500
MercadoPago fees (4.99% de ingresos)	Variable	Variable
Backups R2	\$0.10	ARS 120
Email SendGrid (free tier)	\$0	\$0
UptimeRobot (free)	\$0	\$0
Sentry (free tier)	\$0	\$0
Cloudflare Tunnel (free)	\$0	\$0
Total fijo mensual	\$16.34	ARS 19.608

15.3. Cálculo de fee MercadoPago por plan

Plan	Precio USD	Precio ARS	Fee MP (4.99%)	Neto MP
Base	\$9	ARS 10.800	ARS 539	ARS 10.261
Base + Señales	\$14	ARS 16.800	ARS 838	ARS 15.962
Base + Mentoría	\$29	ARS 34.800	ARS 1.737	ARS 33.063
Combo completo	\$34	ARS 40.800	ARS 2.036	ARS 38.764

16. Proyecciones a 50 personas

16.1. Escenario conservador (40% conversion)

Mes	Clientes	Mix	Ingreso/mes	Costos	Ganancia
1-2	0-3	Trial / Base	\$0-30	\$16	-\$16 a +\$14
3-4	5-10	50% base / 30% señales / 20% mentoría	\$50-100	\$19	+\$31 a +\$81
5-6	15-25	50% / 30% / 20%	\$150-250	\$25	+\$125 a +\$225
7-9	30-40	45% / 35% / 20%	\$300-400	\$35	+\$265 a +\$365
10-12	45-50	40% / 40% / 20%	\$450-500	\$45	+\$405 a +\$455

16.2. Escenario realista (60% conversion)

Mes	Clientes	Ingreso/mes	Costos	Ganancia
12	50	\$625	\$50	+\$575

16.3. Escenario optimista (80% conversion)

Mes	Clientes	Ingreso/mes	Costos	Ganancia
12	50	\$750	\$55	+\$695

16.4. Análisis de sensibilidad

Si el precio es...	Y tenés 50 clientes pagando...	Ganancia mensual
\$5/mes	50	\$226
\$9/mes (base)	50	\$420

Si el precio es...	Y tenés 50 clientes pagando...	Ganancia mensual
\$15/mes (promedio)	50	\$720
\$20/mes	50	\$980

17. Plan de contingencia

17.1. Si el VPS se cae

Síntoma	Acción
UptimeRobot alerta	Verificar SSH y reiniciar servicios
Server no responde	Contactar soporte Hostinger 24/7
Disco lleno	Limpiar logs, aumentar storage
OOM (out of memory)	Agregar swap, optimizar PHP-FPM

17.2. Si un cliente reporta problema de pago

- Verificar en panel de MercadoPago
- Revisar logs del webhook
- Activar manualmente la suscripción si el pago se procesó
- Reembolsar si corresponde (desde MercadoPago)

17.3. Si la DB se corrompe

- Detener el server
- Restaurar último backup de R2
- Verificar integridad
- Reanudar servicio

17.4. Si un cliente quiere cancelar

- MercadoPago cancela la suscripción automáticamente
- Webhook notifica al backend

- Marcar `subscription_status = 'cancelled'`
- Mantener acceso hasta fin del período pagado
- Después: `subscription_status = 'expired'`

18. Cronograma de 12 meses

Mes 1-2: Setup + Lanzamiento

Semana	Actividad
S1	Comprar VPS, dominio, configurar DNS
S2	Deploy código, configurar HTTPS
S3	Integrar MercadoPago
S4	Implementar trial 14 días
S5	Setup backups, monitoreo
S6	Testing E2E con 3 amigos
S7	Crear materiales de marketing (landing, FAQ)
S8	Lanzamiento público, primeros 5 clientes

Mes 3-4: Validación

- 5-10 clientes pagos
- Iterar producto según feedback
- Mejorar onboarding
- Implementar programa de referidos (1 mes gratis por referido)

Mes 5-6: Escalar

- 15-25 clientes
- Marketing orgánico (redes, YouTube)
- Partnerships con influencers de trading
- Análisis de cohortes y churn

Mes 7-9: Optimizar

- 30-40 clientes
- A/B test pricing
- Automatizar más flujos
- Considerar upgrade de VPS si es necesario

Mes 10-12: Escalar más

- 45-50 clientes
 - Programa de afiliados
 - Contenido premium
 - Expansión a otros mercados (LATAM)
-

19. Checklist de implementación

Pre-deploy

- ☐ Comprar VPS KVM 2 (Hostinger)
- ☐ Comprar dominio .app
- ☐ Configurar DNS en Cloudflare
- ☐ Crear cuenta Cloudflare
- ☐ Crear cuenta MercadoPago (developers)
- ☐ Crear cuenta SendGrid (opcional)
- ☐ Crear cuenta UptimeRobot
- ☐ Crear cuenta Sentry
- ☐ Crear bucket R2 para backups

Setup server

- ☐ SSH al VPS con usuario deployer
- ☐ Instalar PHP 8.4 + extensiones
- ☐ Instalar Nginx
- ☐ Instalar MariaDB
- ☐ Instalar Composer
- ☐ Instalar Node.js
- ☐ Hardening SSH (deshabilitar root)

Deploy

- ☐ Clonar repo
- ☐ `composer install --no-dev`
- ☐ `npm install && npm run build`
- ☐ Crear `.env.local` con variables de producción
- ☐ Crear DB y usuario MySQL
- ☐ `doctrine:migrations:migrate`
- ☐ `cache:clear --env=prod`
- ☐ `cache:warmup --env=prod`
- ☐ Permisos `var/` correctos

HTTPS

- ☐ Instalar cloudflared
- ☐ `cloudflared tunnel login`
- ☐ `cloudflared tunnel create`
- ☐ Configurar `/etc/cloudflared/config.yml`
- ☐ `cloudflared service install`
- ☐ Configurar Nginx
- ☐ SSL/TLS en Cloudflare: Full
- ☐ Always Use HTTPS: ON

Integraciones

- ☐ Configurar MercadoPago (Access Token, Public Key)
- ☐ Implementar controller de checkout
- ☐ Implementar webhook controller
- ☐ Crear página de success/failure/pending
- ☐ Configurar email transaccional (SendGrid)
- ☐ Templates de email (bienvenida, recordatorios trial, etc)

Trial system

- ☐ Crear migración para campos de trial
- ☐ `doctrine:migrations:migrate`
- ☐ Implementar TrialService
- ☐ Crear cron job de recordatorios
- ☐ Templates de email día 7, 3, 1, 0

Backups

- ☐ Script `/usr/local/bin/backup-tnsvt.sh`
- ☐ Configurar rclone para R2
- ☐ Cron diario 3 AM
- ☐ Test de restore

Monitoreo

- ☐ UptimeRobot: monitor <https://tnsvt.app>
- ☐ Sentry: DSN configurado
- ☐ Health endpoint `/api/health`
- ☐ Alertas por email/Telegram

APK

- ☐ Cambiar URLs a tnsvt.app
- ☐ `build_apk.bat release`
- ☐ Crear release en GitHub v1.4.0
- ☐ Subir APK
- ☐ Test instalación + login + pago

Marketing

- ☐ Landing page en tnsvt.app/landing
- ☐ FAQ
- ☐ Términos y Condiciones
- ☐ Política de Privacidad
- ☐ Botón de WhatsApp para contacto
- ☐ Invitar a primeros 10 amigos

20. Glosario técnico

Término	Significado
VPS	Virtual Private Server - servidor virtual privado
KVM	Kernel-based Virtual Machine - tipo de virtualización
SSH	Secure Shell - acceso remoto seguro a servidores

Término	Significado
Nginx	Servidor web/proxy reverso
PHP-FPM	FastCGI Process Manager para PHP
Composer	Gestor de dependencias de PHP
OPcache	Cache de bytecode de PHP para mejor performance
Symfony	Framework PHP para aplicaciones web
Doctrine	ORM (Object-Relational Mapper) para PHP
MySQL/MariaDB	Base de datos relacional
Cloudflare	CDN + proxy + seguridad web
Cloudflare Tunnel	Túnel encriptado sin abrir puertos
MercadoPago	Pasarela de pagos de MercadoLibre
SDK	Software Development Kit
Webhook	Notificación HTTP automática entre servicios
APK	Android Package Kit - archivo de instalación Android
PWA	Progressive Web App
FCM	Firebase Cloud Messaging - push notifications
R2	Cloudflare R2 - storage S3-compatible
Cron	Programador de tareas de Linux
GDPR/LOPD	Regulación de protección de datos personales
MRR	Monthly Recurring Revenue - ingreso mensual recurrente
ARPU	Average Revenue Per User - ingreso promedio por usuario
Churn	Tasa de cancelación de clientes
LTV	Lifetime Value - valor de vida del cliente

Término	Significado
CAC	Customer Acquisition Cost - costo de adquirir un cliente

Aprobación

Rol	Nombre	Fecha
Product Owner	Federico Casal	Junio 2026
Tech Lead	(a designar)	-
DevOps	(a designar)	-

FIN DEL DOCUMENTO